python interview questions:

1. What are the key features of Python?
Python is an interpreted, high-level, dynamically-typed programming language with features like simplicity, readability, portability, and support for both OOP and functional programming.

2. How does Python handle memory management?
Python has an inbuilt garbage collector, which manages memory by deallocating objects that are no longer in use. It uses reference counting and a cyclic garbage collector for memory management.

3. What are Python namespaces?
Namespaces in Python are containers that hold names (variables and functions) and map them to objects. There are four types: built-in, global, local, and enclosed namespaces.

4. What is the difference between lists and tuples in Python?
Lists: Mutable, meaning they can be changed after creation.
Tuples: Immutable, meaning they cannot be modified once created.

5. What are Python decorators?
Decorators are a way to modify or enhance the behavior of a function or class without permanently modifying it. They are defined using the @ symbol before a function definition.

6. Explain list comprehension with an example.
List comprehension is a concise way to create lists using an expression and a loop. For example:

```
squares = [x**2 for x in range(5)]
```

7. What is the purpose of self in Python?
self is a reference to the current instance of a class. It is used to access the class variables and methods from within the class.

8. What is the difference between _init_ and _new_ methods?
init: Initializes an instance after it is created. It is the constructor of a class.
_new: Creates a new instance of a class and is called before __init_.

9. What are the different types of inheritance in Python?
Python supports:

Single inheritance
Multiple inheritance
Multilevel inheritance
Hierarchical inheritance
Hybrid inheritance

10. How does exception handling work in Python?
Exceptions in Python are handled using try, except, else, and finally blocks. The try block contains the code that might throw an exception, and the except block handles it.

11. What is the difference between append() and extend() in Python?
append(): Adds an element to the end of a list.
extend(): Adds multiple elements (from another iterable) to the end of a list.

12. How is multithreading achieved in Python?
Multithreading in Python is achieved using the threading module. However, due to Python's Global Interpreter Lock (GIL), threads cannot run in parallel in CPU-bound tasks, but they are useful for I/O-bound tasks.

13. What are Python generators?
Generators are a type of iterable that return items lazily, meaning they generate values one at a time and only when required. They are created using yield instead of return in a function.

14. What is a lambda function in Python?
A lambda function is a small, anonymous function defined using the lambda keyword. It can have any number of arguments but only one expression. For example:

```
square = lambda x: x**2
```

15. How do you perform static type checking in Python?
Python 3.5 introduced type hints that allow static type checking. You can specify types using function annotations, and tools like mypy can check type correctness.

```
def add(x: int, y: int) -> int:
    return x + y
```

16. What is the difference between sort() and sorted()?
sort(): Modifies the list in place.
sorted(): Returns a new sorted list from the original iterable without modifying it.

17. How do you implement data structures like stacks and queues in Python?
Stack: Implemented using lists with append() to push and pop() to remove.
Queue: Implemented using the deque class from the collections module for efficient FIFO operations.

18. When should you prefer tuples over lists?
Ans. Tuples should be used when the data is fixed and does not need to be changed, whereas lists should be used when the data is likely to be modified.

19. Which is better, Django or Flask?
Ans. Both frameworks have their own advantages. Django is a high-level, monolithic framework that requires a thorough understanding of its components, making it suitable for larger projects.

Flask is a micro-framework that is lightweight and better for smaller applications or scripts. My personal preference is Django due to my extensive experience with it.

20. Which libraries do you use in Python?
Ans. I use several libraries, including requests, os, json, numpy, and pandas.

21. What is the purpose of the with keyword in file handling?
Ans.The with keyword ensures that resources are properly managed and automatically closed after their usage, which eliminates the need to manually close the file.

22. What are the most commonly used Git commands?
Ans. Some commonly used Git commands are:

git init: Initializes a new Git repository.
git pull: Pulls updates from a remote repository.
git checkout: Switches branches.
git add: Stages changes.
git commit: Commits staged changes.

23. What is the difference between deepcopy and copy in Python?
Ans. copy.copy() creates a shallow copy of an object. It copies the object itself, but not the objects it refers to.
copy.deepcopy() creates a deep copy of an object. It copies the object as well as all the objects it refers to, recursively.

24. How does Python manage memory?
Ans. Python manages memory using a private heap space. The Python memory manager handles the allocation of heap space for Python objects. The built-in garbage collector is used to recycle unused memory to make it available for heap space.

25. What are Python iterators and generators?
Ans. Iterators: An iterator is an object that contains a countable number of values and can be iterated upon, meaning it returns one value at a time.
Generators: Generators are a simple way of creating iterators using a function and the yield statement. Each call to yield produces a value and suspends the function's state, which is resumed when the next value is requested.

26. What is list comprehension in Python?
Ans. List comprehension provides a concise way to create lists. It consists of brackets containing an expression followed by a for clause. Example:

squares = [x**2 for x in range(10)] print(squares) # Output: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

27. Explain the difference between '==' and 'is' operators.
Ans. == checks for value equality. It compares the values of both operands.
is checks for reference equality. It compares the memory locations of both operands.

```
a = [1, 2, 3] b = a
c = a[:]

print(a == b) # Output: True
print(a is b) # Output: True
print(a == c) # Output: True
print(a is c) # Output: False
```

28. How do you create a dictionary in Python?

Ans. A dictionary in Python can be created using curly braces { } with key-value pairs, or by using the dict() function:

```
# Using curly braces
my_dict = {'name': 'John', 'age': 30}

# Using dict() function
my_dict = dict(name='John', age=30)
```

29. Define iterators in Python?

An iterator is an object that contains a countable number of values. in Python, an iterator is an object which implements the iterator protocol, which consist of the methods _iter() and __next_()

30. How does a function return values?

A return is a value that a function returns to the calling script or function when it completes its task. Adding a description for the return, helps you and anyone else using your function to determine exactly what the value should be used for within the calling script or function.

31. Define slicing in Python?

The slice() function returns a slice object. A slice object is used to specify how to slice a sequence. You can specify where to start the sl

32. What are the two major loop statements?

The group of statements being executed repeatedly is called a loop. There are two loop statements in Python: for and while.

33. What happens when a function doesn't have a return statement? Is this valid?

A function without an explicit return statement returns None . In the case of no arguments and no return value, the definition is very simple. Calling the function is performed by using the call operator () after the name of the function.

34. What happens when a function doesn't have a return statement? Is this valid?

A function without an explicit return statement returns None . In the case of no arguments and no return value, the definition is very simple. Calling the function is performed by using the call operator () after the name of the function.

35. Define package in Python?

Packages are namespaces which contain multiple packages and modules themselves. They are simply directories, but with a twist. Each package in Python is a directory which MUST contain a special file called _init_.py .

36. How can you manage memory in Python?

Know that a private heap in Python holds all the objects. The Python memory manager is the tool that manages this private heap by allocating and reallocating memory. The memory manager comes with many vital components. We can perform many tasks, such as sharing, reallocation, segmentation, and caching, with the help of these components.

37. How are NumPy arrays advantageous over Python lists?

Numpy arrays are easy to use than Python lists. Also, the execution speed of NumPy arrays is faster than Python lists. Moreover, NumPy arrays consume less memory and provide a mechanism to specify data types.

38. What is a stack in Python?

A stack is a linear data structure that follows the Last In, First Out (LIFO) principle, meaning the last element added to the stack is the first one to be removed. You can think of it like a stack of plates; you add and remove plates from the top.
In Python, you can implement a stack using a list, where the end of the list represents the top of the stack.
Basic Operations:
Push: Add an element to the top of the stack.
Pop: Remove and return the top element from the stack.
Peek: Return the top element without removing it.

39. Explain the concept of a queue in Python.

A queue is a linear data structure that follows the First In, First Out (FIFO) principle. In a queue, the first element added is the first one to be removed, similar to a line of people waiting at a checkout.
Key operations:
Enqueue: Add an item to the rear of the queue.
Dequeue: Remove and return the item at the front of the queue.
Front: Return the front item without removing it.
Implementation in Python: Queues can be implemented using a list, but using the collections.deque class is more efficient for performance reasons.

40. What is a generator in Python?

A generator in Python is a special type of iterator that allows you to iterate through a sequence of values lazily, meaning it generates values on-the-fly and yields them one at a time. This is useful for working with large datasets, as it allows you to save memory by not storing the entire sequence in memory at once.

41. Explain the concept of a decorator in Python.

A decorator in Python is a design pattern that allows you to modify the behavior of a function or class. Decorators are often used for logging, access control, memoization, and modifying input/output. They are implemented as functions that take another function as an argument and return a new function that enhances or alters the behavior of the original function.

42. How do you use the map() function in Python?

The map() function in Python is a built-in function that applies a given function to each item of an iterable (like a list or a tuple) and returns a map object (which can be converted to a list or other iterable types). It's often used for transforming data.
Example:

```
def square(x):
    return x * x

numbers = [1, 2, 3, 4, 5]
squared_numbers = list(map(square, numbers))
print(squared_numbers)  # Outputs: [1, 4, 9, 16, 25]
```

43. What is the filter() function in Python?

The filter() function in Python is a built-in function that constructs an iterator from elements of an iterable for which a function returns true. It's commonly used for filtering data based on certain criteria.

Example:

```
def is_even(x):
    return x % 2 == 0

numbers = [1, 2, 3, 4, 5]
even_numbers = list(filter(is_even, numbers))
print(even_numbers)  # Outputs: [2, 4]
```

44. What is a module in Python?

A module in Python is a file that contains Python code, which can include functions, classes, variables, and runnable code. Modules allow you to organize your code into manageable and reusable pieces, making it easier to maintain and understand. By grouping related functionality together, modules promote code reusability.

Example: Suppose you have a file named math_operations.py that contains various mathematical functions. This file acts as a module.

45. How do you import a module in Python?

You can import a module in Python using the import statement. There are several ways to import a module:

Basic Import: Import the entire module.

```
import math
print(math.sqrt(16))  # Outputs: 4.0
```

46. What is a package in Python?

A package in Python is a way of organizing multiple related modules into a single directory hierarchy. It is a directory that contains a special _init_.py file (which can be empty) that signifies to Python that the directory should be treated as a package. Packages enable a structured way to manage and organize modules, especially in larger projects.

47. Explain the concept of namespace in Python.

A namespace in Python is a container that holds a set of identifiers (names) and the corresponding objects they refer to. It ensures that names are unique and can be used without confusion. Namespaces help prevent naming conflicts by allowing the same name to be used in different contexts.
There are several types of namespaces in Python:
Built-in Namespace: Contains built-in functions and exceptions.
Global Namespace: Contains names defined at the top level of a module.
Local Namespace: Contains names defined within a function.
When you reference a name, Python searches through the namespaces in a specific order (local, enclosing, global, built-in) to find the corresponding object.

48. How do you create a package in Python?

To create a package in Python, follow these steps:
Create a Directory: Create a new directory that will contain your package.
Add _init.py: Inside the directory, create an __init_.py file. This file can be empty or can contain initialization code for the package.
Add Modules: Add one or more module files (e.g., .py files) to the package directory.

49. What is a generator in Python?

A generator in Python is a special type of iterator that allows you to iterate through a sequence of values lazily, meaning it generates values on-the-fly and yields them one at a time. This is useful for working with large datasets, as it allows you to save memory by not storing the entire sequence in memory at once.

50. Explain the concept of a decorator in Python.

A decorator in Python is a design pattern that allows you to modify the behavior of a function or class. Decorators are often used for logging, access control, memoization, and modifying input/output. They are implemented as functions that take another function as an argument and return a new function that enhances or alters the behavior of the original function.

60. Explain the concept of polymorphism in Python.

Polymorphism is an OOP concept that allows objects of different classes to be treated as objects of a common superclass. It enables a single interface to represent different underlying forms

(data types). In Python, polymorphism is often achieved through method overriding and operator overloading.

61. What is inheritance in Python?

Inheritance is a feature of OOP that allows one class (child class) to inherit attributes and methods from another class (parent class). This promotes code reuse and can simplify the creation of new classes.

62. How do you define a class in Python?

To define a class in Python, you use the class keyword followed by the class name and a colon. The class body is defined with an indentation.
Example:

```
class Car:
    def _init_(self, brand, model):
        self.brand = brand
        self.model = model

    def display_info(self):
        return f"{self.brand} {self.model}"
```

In this example, the Car class has a constructor to initialize the brand and model attributes and a method to display information about the car.

63. Explain the concept of classes and objects in Python.

Classes: A class is a blueprint or template for creating objects. It defines the attributes (data) and methods (functions) that the objects created from the class will have. Classes can also include special functions called constructors, which are used to initialize the attributes of the object.

Objects: An object is an instance of a class. When a class is defined, no memory is allocated until an object of that class is created. Each object can have different attribute values while sharing the same methods defined in the class.

64. What are the different types of function arguments in Python?

Python supports several types of function arguments:
Positional Arguments: These are the most common type of arguments, where the values are passed to the function in the order they are defined.
Keyword Arguments: You can specify arguments by name, allowing you to pass them in any order.
Default Arguments: You can provide default values for parameters. If a value is not passed for that parameter, the default value is used.
Variable-length Arguments: These allow you to pass a variable number of arguments to a function using *args for positional arguments and **kwargs for keyword arguments.

65. Explain the return statement in Python.

The return statement is used to exit a function and send a value back to the caller. If no return statement is specified, the function will return None by default.

Example:

```
def square(number):
    return number ** 2

result = square(4)  # result is 16
```

66. How do you use the lambda function in Python?

A lambda function is a small anonymous function defined using the lambda keyword. It can take any number of arguments but can only have one expression. Lambda functions are often used for short, throwaway functions and are commonly used in higher-order functions like map(), filter(), and sorted().
Example:

```
# A simple lambda function to add two numbers
add = lambda x, y: x + y
result = add(5, 3)  # result is 8

# Using lambda with filter() to get even numbers
numbers = [1, 2, 3, 4, 5, 6]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))  # Output: [2, 4, 6]
```

67. What is the difference between the read() and readline() methods?

read(): This method reads the entire contents of the file as a single string. It can take an optional argument specifying the number of bytes to read. If no argument is provided, it reads until the end of the file.

readline(): This method reads a single line from the file. Each time you call readline(), it reads the next line, allowing you to iterate through the file line by line.

68. Explain the concept of file modes in Python.

File modes in Python determine how a file is opened. The most common modes include:

'r': Read mode (default). Opens the file for reading. An error occurs if the file does not exist.
'w': Write mode. Opens the file for writing, creating it if it does not exist or truncating the file to zero length if it does.
'a': Append mode. Opens the file for writing, adding new data at the end of the file without truncating it.
'b': Binary mode. Opens the file in binary format (e.g., for images or executable files). This mode can be combined with others (e.g., 'rb', 'wb').
'x': Exclusive creation mode. Opens the file for writing, but fails if the file already exists.
't': Text mode (default). Opens the file in text format. This mode can also be combined with others (e.g., 'rt', 'wt').

69. How do you raise an exception in Python?

You can raise an exception in Python using the raise statement. This can be used to trigger an exception manually, either with a specific exception type or with a custom message.

Example:

```
age = -1
if age < 0:
    raise ValueError("Age cannot be negative.")
```

In this example, a ValueError is raised if the age is negative, allowing you to handle this situation in a controlled manner.

70. Explain the concept of list comprehension in Python.

Highlight your knowledge of concise and efficient coding techniques by discussing list comprehension. Explain that list comprehension provides a compact way to create lists based on existing lists or other iterable objects. Showcase a simple example, such as generating a new list of squares from an existing list of numbers.

71. Discuss the difference between shallow copy and deep copy.

Demonstrate your understanding of object references and memory management by explaining the concepts of shallow copy and deep copy. Clarify that a shallow copy creates a new object but references the same elements, while a deep copy creates a new object and recursively copies all elements and nested objects.

72. What is the difference between a module and a package in Python?

A9. A module is a single file containing Python code, while a package is a collection of modules and sub-packages organized in a directory hierarchy. Packages are used to structure large Python projects, allowing for better code organization and reusability.

73. How can you remove duplicate elements from a list in Python?

A. You can remove duplicate elements from a list by converting it to a set and then back to a list.
Example:

Code Implementation

```
list1 = [1, 2, 2, 3, 4, 4, 5]
unique_list = list(set(list1))
print(unique_list)  # Output: [1, 2, 3, 4, 5]
```

74. How can Python be an interpreted language?

Python is called an interpreted language because it goes through an interpreter, which turns code you write into the language understood by your computer’s processor.

75. Define slicing in Python?

The slice() function returns a slice object. A slice object is used to specify how to slice a sequence. You can specify where to start the slicing, and where to end. You can also specify the step, which allows you to e.g. slice only every other item.

76. Define package in Python?

Packages are namespaces which contain multiple packages and modules themselves. They are simply directories, but with a twist. Each package in Python is a directory which MUST contain a special file called _init_.py .

- Data types in Python specify the type of data a variable can hold, such as int, float, str, list, tuple, set, and dict.
- A list is mutable, whereas a tuple is immutable.
- Slicing is used to access a range of elements from a list or tuple using [start:stop] or [start:stop:step].

ABOUT GIT:

- Git is a distributed version control system that tracks changes in files and allows multiple developers to collaborate on software projects.
- .To resolve a conflict in Git, manually edit the conflicting files, remove conflict markers, and then commit the changes.
- A conflict in a file occurs when Git cannot automatically merge changes from different branches or commits due to conflicting modifications in the same part of the file.
- Git is unable to merge when changes to the same lines of a file are made in different branches, and it can't automatically determine which version to keep.
- This means that if two different branches have modifications in the same section of a file, Git cannot decide which change to apply, so it flags a conflict for manual resolution.
- Two branches refer to separate versions of a repository where each branch can have different changes or features, allowing for parallel development.

Python is a high-level, interpreted programming language.
It’s known for:
Simple syntax (easy to learn)
Versatility (web, automation, data science, AI)
Being open-source and cross-platform

Yes, Python supports OOP (Object-Oriented Programming).
It includes:
Classes and Objects
Inheritance
Polymorphism
Encapsulation
Abstraction

OOP features in Python:

- *Class & Object* – Blueprint and instance of data/behavior
- *Encapsulation* – Hiding internal details using private variables
- *Inheritance* – Reusing code from a parent class
- *Polymorphism* – Same function name behaves differently
- *Abstraction* – Hiding complex logic, showing only essentials

- *List* is mutable (can be changed)
- *Tuple* is immutable (cannot be changed)
- Lists use more memory, tuples are faster
- Lists: [], Tuples: ()
- Tuples are safer for fixed data

Methods:
Lists have many built-in methods like append(), remove()
Tuples have fewer methods, mainly count() and index()

Project:
Small Project:1

Read name and marks of 5 students from the console , write into text file marks.txt
read this file marks.txt and print the output in the following format
if marks >=50 pass , otherwise fail

```
**********************************************************************
name    marks    result
**********************************************************************
***     65      pass
***     35      fail
**********************************************************************
```

Small Project:2
Read name ,m1 and m2 marks of 5 students from the console , write into text file marks.txt
read this file marks.txt and print the output in the following format
if marks >=50 pass , otherwise fail

```
**********************************************************************
name    m1    m2    total
**********************************************************************
***     65    55    120
***     35    45    80
**********************************************************************
```

Project:
Project: Student Grade Processor
Features:
Uses map() to convert numerical scores into letter grades.
Uses filter() to find students who passed (grade above 'F').
Displays the final list of students with their grades.

```
if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    elif score >= 70:
        return 'C'
    elif score >= 60:
        return 'D'
    elif score >= 50:
        return 'E'
    else:
        return 'F'
# Define student data (name, score)
students = [
    ("Alice", 85),
    ("Bob", 42),
    ("Charlie", 73),
    ("David", 90),
    ("Eve", 55),
    ("Frank", 30)
]
```

output:

```
All Students with Grades:
('Alice', 'B')
('Bob', 'F')
('Charlie', 'C')
('David', 'A')
('Eve', 'E')
('Frank', 'F')
Passing Students:
('Alice', 'B')
('Charlie', 'C')
('David', 'A')
('Eve', 'E')
```

Student Grade Processor – Simple Interview Explanation
Step 1: Create Student Data
I started by creating a list of students with their names and marks.
python
Copy
Edit
[("Alice", 85), ("Bob", 42), ...]
Step 2: Write a Grading Function
I made a function that takes marks and returns a letter grade like A, B, C, etc.

Example: 90+ is A, 80–89 is B, and so on.

Step 3: Use map() to Convert Scores to Grades
I used the map() function to apply the grading function to each student.

This gives a new list with student names and their letter grades.

Step 4: Use filter() to Find Passing Students
I used filter() to select only the students who did not get grade F.

These are the passing students.

Step 5: Print the Results
I printed:

All students with their grades.

Then, only the students who passed.

🧠 What It Shows
My ability to use map() and filter() effectively.

Clean and structured data handling using functions and tuples.

Simple but real-world logic for academic tasks